BÁO CÁO KẾT QUẢ THỰC HÀNH

Môn học: Lập trình Java Web Application

Tên dự án: HỆ THỐNG QUẢN LÝ MƯỢN/TRẢ THIẾT BỊ IT & PHÒNG LAB

Họ và tên sinh viên: Lê Đức Anh

Lớp: CNTT 2 – Nhóm: 2

I. NỘI DUNG THỰC HÀNH

Trong dự án này, em đã xây dựng hệ thống Web Portal nội bộ hỗ trợ sinh viên đăng ký mượn thiết bị và phòng Lab. Ứng dụng được phát triển trên nền tảng **Spring WebMVC 7.x** thuần, tập trung vào việc xử lý form phức tạp và kiểm soát logic nghiệp vụ chặt chẽ.

1. Kiến trúc hệ thống (Layered Architecture):

- **Configuration:** Thiết lập hoàn toàn bằng **Java Config** (thay cho XML), cấu hình DispatcherServlet, ThymeleafViewResolver và đăng ký Validator để xử lý dữ liệu.
- **Controller:** * Sử dụng @GetMapping để hiển thị danh mục thiết bị và form đăng ký.
    - Sử dụng @PostMapping kết hợp với @Valid và BindingResult để xử lý đăng ký mượn thiết bị.
- **Service & Repository:** * Tầng Service xử lý các quy tắc nghiệp vụ (Business Rules) như kiểm tra tồn kho, so sánh ngày mượn/trả.
    - Tầng Repository quản lý danh sách thiết bị/phòng Lab mẫu (Mock Data) với các thông tin về số lượng khả dụng.
- **Model & Validation:** Định nghĩa các Class POJO với các Annotation như @NotBlank, @Min, @Email, và @Future để kiểm soát tính toàn vẹn của dữ liệu ngay từ tầng đầu vào.

2. Xây dựng Giao diện (View Layer):

- Sử dụng **Thymeleaf Fragment** để đồng bộ hóa Header và Footer.

- Sử dụng thư viện CSS để thiết kế Form đăng ký chuyên nghiệp, hỗ trợ hiển thị thông báo lỗi ngay dưới từng trường dữ liệu bằng th:errors.
- Tích hợp **RedirectAttributes** để truyền dữ liệu trạng thái (Flash Attributes) sau khi đăng ký thành công.

3. Xử lý Logic nghiệp vụ (Business Logic):

- **Logic thời gian:** Kiểm tra đảm bảo ngày nhận phải sau ngày hiện tại và ngày trả phải sau ngày nhận.
- **Logic tồn kho:** Đối soát số lượng sinh viên muốn mượn với số lượng thực tế còn lại trong hệ thống.
- **Xử lý lỗi (Error Handling):** Áp dụng cơ chế giữ lại dữ liệu đã nhập trên form khi có lỗi xảy ra, giúp tăng trải nghiệm người dùng.

II. KẾT QUẢ ĐẠT ĐƯỢC

- **Về kiến thức:** Hiểu sâu về vòng đời Request trong Spring MVC thuần. Thành thạo cơ chế **Data Binding** và cách tích hợp **Hibernate Validator** để bắt lỗi dữ liệu.
- **Sản phẩm:** Xây dựng thành công hệ thống Lab Management với các tính năng:
    - Hiển thị danh sách thiết bị/phòng Lab kèm trạng thái và số lượng tồn.
    - Form đăng ký mượn thiết bị với đầy đủ các ràng buộc về Email, Mã sinh viên (Regex), và số lượng.
    - Hệ thống quản lý phía Admin cho phép xem danh sách các yêu cầu mượn đã gửi.
    - Thông báo "Đăng ký thành công" thông qua cơ chế Flash Attributes khi chuyển hướng trang.
- **Kỹ năng:** Thành thạo việc cấu hình Spring không dùng Spring Boot, kỹ năng xử lý logic ngày tháng trong Java và hiển thị lỗi trên giao diện Thymeleaf.

Link github:
https://github.com/Leducanh30112006/PTIT_CNTT2_IT210_SS10

III. KHÓ KHĂN VÀ CÁCH KHẮC PHỤC

- **Khó khăn:** Việc so sánh logic giữa hai trường (ngày nhận và ngày trả) không thể thực hiện bằng các Annotation đơn lẻ thông thường.
    - **Cách khắc phục:** Triển khai thêm logic kiểm tra tại tầng Service và sử dụng bindingResult.rejectValue để đẩy lỗi thủ công về giao diện khi logic ngày tháng không hợp lệ.
- **Khó khăn:** Gặp lỗi 404 khi cấu hình DispatcherServlet bằng Java Config do sai sót trong việc đăng ký Resource Handler cho CSS/Images.
    - **Cách khắc phục:** Ghi đè phương thức addResourceHandlers trong WebConfig để chỉ định đúng đường dẫn đến thư mục /static/.

IV. KINH NGHIỆM RÚT RA

- Việc hiểu rõ cấu trúc Spring MVC thuần giúp nắm bắt bản chất của Framework tốt hơn so với việc sử dụng các cấu hình tự động của Spring Boot.
- Áp dụng **PRG Pattern** (Post-Redirect-Get) là yếu tố sống còn trong các ứng dụng Web để đảm bảo dữ liệu không bị trùng lặp và tránh lỗi phía người dùng.
- Tổ chức code theo mô hình Layer giúp việc mở rộng các Custom Validator (như kiểm tra trùng lịch phòng Lab) trở nên dễ dàng và mạch lạc hơn.

V. KẾT LUẬN

Dự án Quản lý thiết bị Lab đã giúp em hoàn thiện kỹ năng xử lý form và kiểm soát dữ liệu chặt chẽ theo tiêu chuẩn SRS. Đây là nền tảng quan trọng để em tiếp tục phát triển các hệ thống quản lý tài nguyên phức tạp hơn, áp dụng các kỹ thuật nâng cao về bảo mật và tối ưu hóa trải nghiệm người dùng trong tương lai.